

TP de probabilités — Estimer π par Monte-Carlo

1. Objectif

Utiliser une simulation aléatoire pour estimer la valeur de π .

Comparer ensuite cette méthode avec une formule très efficace découverte par Ramanujan.

2. Principe mathématique

On considère un carré de côté 2 contenant un disque de rayon 1.

Un point appartient au disque lorsque :

$$x^2 + y^2 \leq 1$$

Aire du carré : _____

Aire du disque : _____

Probabilité qu'un point soit dans le disque :

$$P = \text{_____}$$

On obtient alors :

$$\pi \approx 4 \times \text{fréquence des points dans le disque}$$

3. Simulation à la main

Point	x	y	x^2+y^2	Dans le disque ?
A	0,3	0,4		
B	0,9	0,7		
C	-0,5	0,2		
D	-0,8	-0,6		
E	0,1	-0,9		

Nombre de points dans le disque : _____

Fréquence : _____

Estimation de π : _____

4. Partie pratique Python

4.1 Générer un nombre aléatoire

```
from random import uniform

x = uniform(____, ____)
```



```
print(x)
```

Question 1. Que remarques-tu lorsque le programme est exécuté plusieurs fois ?

4.2 Tester si un point est dans le disque

```
from random import uniform

x = uniform(-1, 1)
y = uniform(-1, 1)

if x**2 + y**2 <= ____:
    print("Dans le disque")
else:
    print("Hors du disque")
```

Question 2. Quelle condition permet de tester si un point appartient au disque ?

4.3 Répéter l'expérience

```
from random import uniform

n = 1000
compteur = 0

for i in range(____):

    x = uniform(-1, 1)
    y = uniform(-1, 1)

    if x**2 + y**2 <= 1:
        compteur = compteur + ____

frequence = compteur / n
pi_estime = 4 * frequence

print(pi_estime)
```

Question 3. À quoi sert la variable compteur ?

Question 4. Tester avec :

- 100 expériences
- 1 000 expériences

- 10 000 expériences
- 100 000 expériences
- 1 000 000 expériences

Nombre d'expériences	Valeur obtenue
100	
1 000	
10 000	
100 000	
1 000 000	

Question 5.

1. La précision augmente-t-elle ?
2. Le calcul est-il rapide ?
3. La méthode semble-t-elle efficace ?

5. Programme final

```

from random import uniform

n = int(input("Nombre de points : "))

points_dans_disque = ____

for i in range(n):

    x = uniform(____, ____ )
    y = uniform(____, ____ )

    if ____ + ____ <= ____:
        points_dans_disque = points_dans_disque + ____

frequence = points_dans_disque / ____
pi_estime = ____ * frequence

print(pi_estime)

```

6. Une méthode beaucoup plus rapide : Ramanujan

Le mathématicien indien Srinivasa Ramanujan a découvert des formules extrêmement efficaces pour calculer π.

Voici une célèbre formule :

$$1/\pi = (2\sqrt{2} / 9801) \times \sum [(4k)! (1103 + 26390k) / ((k!)^4 \times 396^{(4k)})]$$

Quelques termes suffisent pour obtenir beaucoup de décimales exactes.

7. Programme Python — Ramanujan

```
from math import factorial, sqrt

somme = 0

for k in range(3):

    terme = (factorial(4*k) * (1103 + 26390*k)) / \
            ((factorial(k))**4 * 396**(4*k))

    somme = somme + terme

inverse_pi = (2 * sqrt(2) / 9801) * somme

pi_estime = 1 / inverse_pi

print(pi_estime)
```

8. Comparaison des méthodes

Méthode	Principe	Rapidité	Précision
Monte-Carlo			
Ramanujan			

Question 6.

1. Quelle méthode est la plus rapide ?
2. Quelle méthode est la plus précise ?
3. Pourquoi Monte-Carlo reste-t-elle intéressante ?